

ForL0 State Backend 设计说明书

1. 整体架构设计

ForL0 StateBackend 设计的目标是充分利用鲲鹏 CPU 服务器提供的 L0 Cache 特性，通过缓存友好的数据布局与分层索引，将 Flink 状态访问延迟降到最低并提升吞吐与效率。如图 1 所示，整体采用两级索引和键值分离的架构：顶层为 L0 Table（Cache 加速区），针对高频访问做极低延迟响应；底层为 Main Table（主状态表），负责存放全量索引并通过扩展桶机制解决冲突与扩展需求。索引指针指向堆外的 Entry 数据块（Entry Store），实现索引与数据的分离以减少缓存抖动并优化内存布局。此架构配备专门的 Checkpoint 接口模块，确保状态数据的一致性与快速恢复能力。此外，还设计了专门的容错机制适配层，使得本方案能对接 Flink 原生的 Checkpoint 和故障恢复机制，达到高兼容性与易部署性。

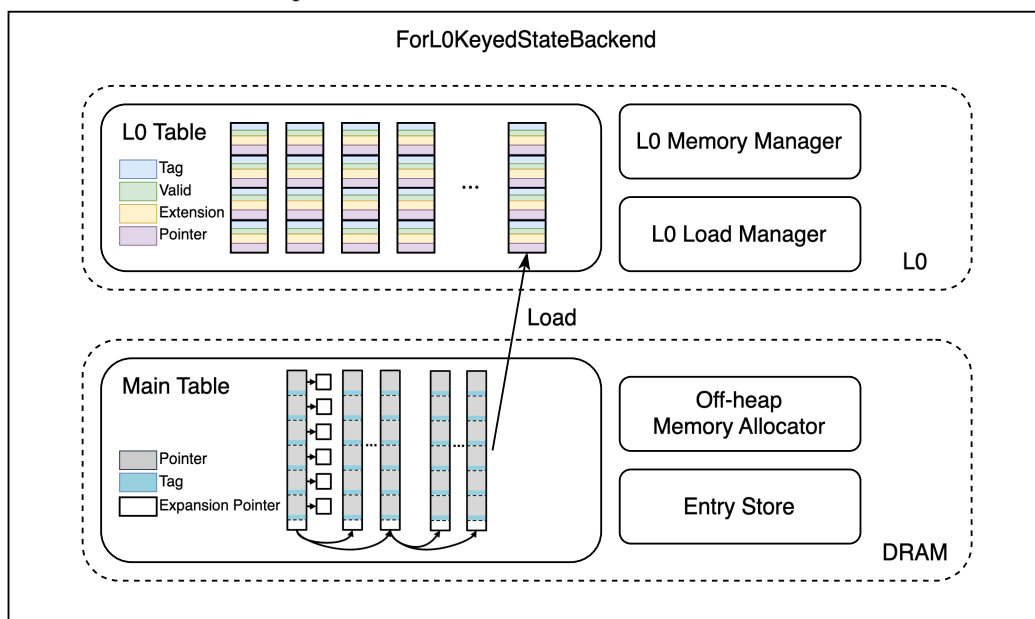


图 1 整体架构图

本系统采用分层设计，分为 L0 Table 和 Main Table 两大核心部分，并配合内存分配管理与负载管理等辅助组件。主要具有以下特点：

- **缓存友好的数据布局：**64 字节缓存行对齐的桶结构，降低缓存行读取开销；键值分离存储，增强空间局部性；
- **双层索引：**先查 L0 Table（快速命中），未命中再落到 Main Table，查到后可将热点回填 L0 以提升后续访问命中率；
- **多种替换策略：**支持 LRU、LFU、FIFO、RANDOM 等基础替换策略以及热点感知的替换策略，便于根据工作负载选择最优策略；
- **堆外内存管理：**使用基于 Flink 内置的 MemoryManager 进行统一内存分配与回收；
- **检查点与容错：**内置适用于堆外内存的 Checkpoint 接口模块、Snapshot 策略与本地恢复配置，并提供容错适配层以对接 Flink 的容错机制，保证一致性与快速恢复；
- **易集成与部署：**面向 Flink 1.19+，Java8+ 构建，将生成 JAR 放入 Flink lib 目录或通过代码方式设置 StateBackend 即可集成，而无需对用户代码做任何修改。

ForL0StateBackend 通过分层设计、缓存对齐的数据结构与统一内存管理，在保持与 Flink 原生接口兼容的同时，显著提升热点状态访问的延迟和整体状态访问效率，并为未来更加充分地利用鲲鹏 L0 硬件能力留出扩展路径。

1.1 L0 Table

L0 Table 为顶层热点索引层，面向极低延迟的读写路径设计，常驻于 L0 Cache 可用的内存预算内（由 L0 Memory Manager 管控）。主要职责是承载高频访问键的索引元信息，实现快速命中、快速回填与最小化对主表访问。

其物理布局由若干哈希桶组成，桶大小按 CPU 缓存行对齐（64B），每桶包含 4 个紧凑 slot，以减少缓存行抖动与跨行读取。其结构如图 2 所示。

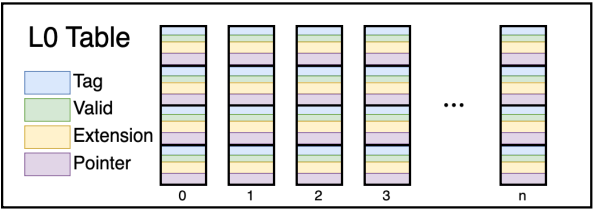


图 2 L0 Table

- Tag**（2B）：对应 key 的摘要，用于快速比对；
- Valid**（1B）：用于标记该 slot 信息是否有效；
- Extension**（5B）：用于对齐 16B，也可留作扩展字段使用（如 LRU 位等）；
- Pointer**（8B）：指向实际键值对的地址；

L0 Table 所占用的内存由专用的 L0 Memory Manager 管理，其使用 JNI 封装了 L0 内存分配接口，而负载和热点感知由 L0 Load Manager 实现，动态提升热点 Key。

1.2 Main Table

Main Table 为全量索引层，承担完整状态索引的长期存储与冲突解决，设计以缓存友好为目标，并通过局部扩展机制尽可能减少全表重哈希次数。

其物理布局同样由若干哈希桶组成，每个桶为 64B 对齐的 Pointer Set，通过扩展指针字段支持局部树型扩展。Main Table 的每个哈希桶包含 6 个 slot 和一个扩展指针，其结构如图 3 所示。

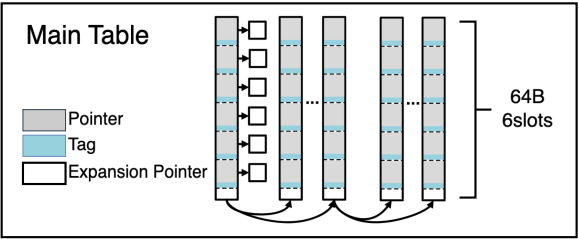


图 3 Main Table

- Pointer**（8B）：指向实际键值对的地址；
- Tag**（2B）：对应 key 的摘要，用于快速比对；
- Expansion Pointer**（4B）：4 个用于树型扩展的指针，每个指针为扩展桶的下标，最多支持扩展 255 个桶；

Main Table 所使用的堆外内存由运行时注入的 Flink Memory Manager 进行分配。当发生冲突时，默认使用树型局部扩展策略，通过 Tag 的低 2 位确定扩展桶并进行扩展，只有当达到全局扩容条件（扩展桶满或达到负载因子阈值）时才进行全局扩容。

1.3 内存负载与管理

L0 Table 与 Main Table 各自由独立的内存管理组件负责其内存生命周期、配额控制与回收策略。

Main Table 使用运行时注入的 Flink MemoryManager 分配的堆外内存区域，采用 MemoryManagerAllocator 做统一记账与配额约束，针对堆外读写使用 Unsafe 或等效低开销 API 以减少复制和序列化开销；L0 映射区的内存由专用的 L0 Memory Manager 通过 JNI 封装的系统调用接口分配与管理，负责映射区的分配/释放以及 pin/unpin 语义。

Entry Store 作为键值对的物理存储层，采用大块（chunk/slab）分配、分级空闲列表与位图等高效数据结构来减少碎片并加速空闲查找。所有关键内存操作均暴露可观测指标（内存占用、分配/释放速率、碎片率、pin 数量、驱逐次数等），并提供可配置阈值用于自动触发回收或降级行为，便于运维调优与故障定位。

1.4 组件解耦与集成

L0Table 与 MainTable 在代码层面是独立实现的模块（L0Table、MainTable），二者与 Entry Store 在上层的 ForL0StateMap 中进行组装并协同工作。设计上 MainTable 是一致性与持久化的权威来源，所有写入、删除与扩容操作最终以 MainTable 为准；L0Table 仅承担热点索引的缓存职责，索引回填、驱逐与命中逻辑由可替换的替换策略模块与 L0 管理组件驱动，因此 L0 Table 可以作为一个可选的性能优化配置项向用户提供。

在与 Flink 的集成方面，ForL0KeyedStateBackend 承担组件的组装与生命周期管理，插件化接入通过 StateBackendFactory 注册点完成。运行时以插件形式与 Flink 对接，导入 Jar 包后通过构造参数或配置文件（config.yaml）启用或配置 ForL0 后端，L0 层为可选开关，对 Flink 内核源码以及上层用户代码透明无侵入。

2. 架构实现细节

ForL0 State Backend 基于 Flink 原生 HashMapStateBackend 架构进行改造，在保留原有核心接口和部分可复用组件的基础上，引入了多个新增组件以支持上述架构设计。这些新增组件与原有 Flink StateBackend 紧密协作，既保持了 API 兼容性，又实现了存储结构和缓存策略的深度优化。其组件关系如图 4 所示。

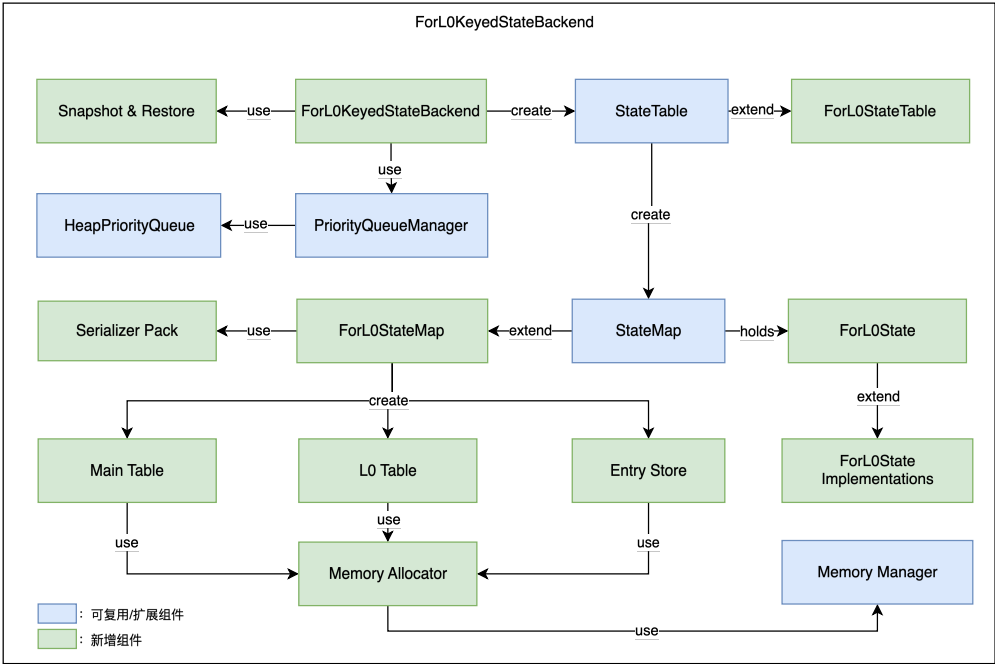


图 4 ForL0StateBackend 组件图

2.1 可复用组件与扩展组件

在 ForL0 State Backend 中，以下 Flink 原生组件可直接复用。

2.1.2 StateTable / StateMap

ForL0 在接口层面复用 Flink 的 StateTable/StateMap 抽象，但用 ForL0 专用实现替换了原有的 CopyOnWriteStateMap。ForL0StateTable / ForL0StateMap 承担对外统一状态表接口（增删改查、迭代、快照/恢复等），对上层 KeyedStateBackend 保持 API 兼容，同时在内部组合 L0Table、MainTable 与 Entry Store 完成双层索引与键值分离的实现。StateTable 抽象仍作为外部契约，便于与 Flink 的 Snapshot/Restore 流程对接。

2.1.3 PriorityQueueManager / HeapPriorityQueue

定时器/优先队列相关功能直接复用或兼容 Flink 自身的堆实现(HeapPriorityQueue等)，ForL0 不改变定时器语义，PriorityQueue 的管理器按原有接口工作，保证与现有事件时间/处理时间定时器逻辑兼容。

2.1.4 ForL0KeyedStateBackend

ForL0KeyedStateBackend 是与 Flink KeyedStateBackend 的适配扩展点，负责在任务初始化时组装 ForL0StateTable、分配后端所需内存、初始化所需管理组件等，并在 Snapshot/Restore 路径中插入 ForL0StateMap 的序列化/反序列化逻辑。该类承担生命周期管理、checkpoints 的协调以及把 ForL0 的内部结构映射到 Flink 的 KeyedState 接口上。

2.1.5 MemoryManager

ForL0StateBackend 通过 MemoryManagerAllocator 向 Flink 的 MemoryManager 请求大块堆外内存，并依赖其配额、回收与异常上报机制。MemoryManager 负责提供提供统一的资源管理功能，分配失败或超额时由 Flink 的内存治理链路处理。

2.2 关键新增组件

2.2.3 ForL0StateMap

ForL0StateMap 是本后端的核心数据结构，实现了双层索引：L0Table 为热点索引缓存，MainTable 为权威索引并支持局部树型扩展。ForL0StateMap 负责所有状态的增删改查逻辑、冲突解决、扩展桶递归插入/查找、以及与 Entry Store 的 KV 读写交互。该组件封装了 L0 回填、驱逐与一致性失效逻辑，确保写操作以 MainTable 为准且能在查询时提升热点至 L0。

2.2.4 Memory Allocator

专用的堆外内存分配器，负责为主表、扩展桶池以及 Entry Store 分配和释放内存块。所有内存均通过 Flink 的 MemoryManager 进行大块分配，并保证 64B 对齐。该组件为 ForL0StateMap 提供统一的内存管理接口。

2.2.5 MainTable

MainTable 实现了缓存友好的主索引结构，每个主桶为 64B 对齐、包含固定 slot 与用于局部扩展的字段。MainTable 负责全量索引的存储、扩展桶分配与回收、全表重分配(resize)以及与 Entry Store 的地址绑定，并采用局部子桶扩展与递归插入来减少全量迁移的次数。

2.2.6 L0Table

L0Table 提供顶层的热点索引缓存，实现了包含多种替换策略的元数据支撑（LRU/LFU/FIFO/RANDOM 等），并通过 ForL0StateMap 的回填/驱逐逻辑与 MainTable 保持最终一致性，且其内容为可失效的缓存，可在恢复后按需重建。

2.2.7 Entry Store

Entry Store 即 State Map 所管理的 KVNode 空间，用于存放序列化后的 key、namespace、value 等变长数据。KVNode 布局保存必要的元信息（长度、标记、指针等），支持快速定位与重用。Entry Store 负责实际条目的增删改操作，并在全表迁移或 checkpoint 恢复时保证物理地址不变以简化索引重建。

2.2.8 ForL0State

在 State 层面，为不同类型的 State 提供了对应的 ForL0 实现（如 ValueState、ListState、MapState、ReducingState 等），这些类在 ForL0KeyedStateBackend 中注册被调用创建与更新。每个 ForL0State 实现将状态操作翻译为对 ForL0StateMap 的具体增删改查操作，从而向上层暴露与 Flink API 完全兼容的行为。

2.2.9 Snapshot & Restore

快照与恢复流程沿用 Flink HashMapStateBackend 的 Snapshot/Restore 语义：MainTable 与 Entry Store 为快照的数据来源，快照过程序列化主索引与 KV 存储，并在恢复时重建 MainTable 索引及 KV 区。Snapshot 路径上插入了必要的序列化逻辑以持久化 MainTable 与 Entry Store 的元信息。

2.2.10 Serializer Pack

SerializerPack 是用于集中管理序列化逻辑的模块，负责持有并复用由 Flink 运行时注入的 TypeSerializer 实例，并对 ForL0StateBackend 中的二进制布局提供统一的序列化/反序列化方法。该组件为 Snapshot/Restore 路径、EntryArena 的持久化与重建、以及 MainTable 索引序列化提供一致的编码/解码接口，同时通过缓存序列化实例和复用缓冲区来降低序列化开销。

2.3 State Map 实现

ForL0StateMap 是 ForL0 State Backend 的核心实现，围绕 L0 Table + Main Table 的双层索引与键值分离思想实现具体的索引与存储逻辑。为控制内存布局和减少 GC，所有索引结构与 KV 存储均分配在堆外内存上并以 64 字节缓存行对齐，索引结构采用固定大小的桶布局以减少缓存抖动，KV 区（Entry Store）则使用变长布局以节省空间并支持任意序列化后的 key/namespace/value。

2.3.1 结构组成

如图 5 所示，ForL0StateMap 由 L0 Table、Main Table 以及 Entry Store 组成：

L0 Table: 负责缓存最活跃的热点 key，保证极低延迟访问。其容量与替换策略可配置，连续存储于一块连续的堆外内存中。

Main Table: 负责存储全量索引，采用每桶多 slot 和局部树型扩展的缓存友好结构，实现高负载、高冲突情况下的稳定高性能。

Entry Store: 所有实际的 key-value 数据存储于专门的堆外内存池中，采用键值分离设计提高缓存友好性。

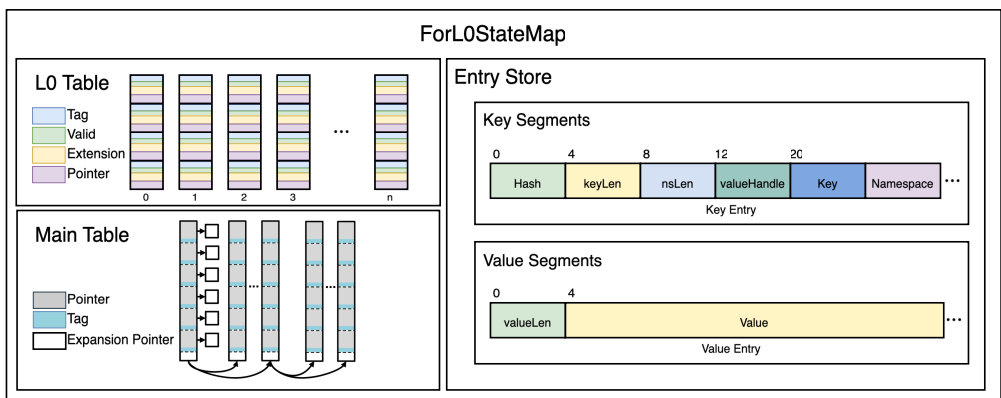


图 5 ForL0StateMap 结构

2.3.2 L0 Table 实现

L0 Table 的实现和内存布局如下：

- **桶结构**：每个 bucket 为 64 字节，包含 4 个 slot，每个 slot 包含 tag、有效位、条目指针、以及预留扩展字段（如 LRU/LFU/版本等），具体说明由表 1 给出；
- **物理分布**：所有 L0 Table 内存块一次性分配，64B 对齐，占用一块连续内存便于批量加载；
- **管理策略**：热点 key 会被动态提升至 L0 Table。插入、更新、查询均会刷新 L0 Table，对应 slot 的选择和替换由可配置的替换算法决定；
- **一致性维护**：L0 Table 仅作缓存，主表持有全量状态索引，写入或扩容等变更后，相关 L0 条目会自动失效或被刷新。

表 1 L0 Table Slot 数据结构

字段	说明	字节数(Byte)
Tag	Key 的哈希摘要	2
Valid	有效位	1
Extension	扩展位/LRU/版本/字节对齐	5
Pointer	指向 KV 项的指针	8

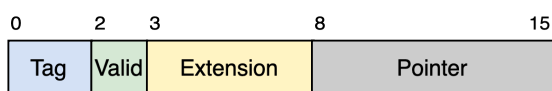


图 6 L0 Table slot 布局

L0 Table 为 L0 区域的桶数组，每个桶为包含 4 个 slot，如图 6 所示，扩展字段部分可灵活利用。

2.3.3 Main Table 设计

Main Table 采用缓存友好设计的树型扩展哈希表，其实现和内存布局如下：

- **桶结构**：每个主表桶（bucket）为 64 字节，包含 6 个 slot 和 4 个子桶指针；slot 保存 key、namespace 的 tag 及指向实际存储条目的指针，所有 slot 均 64B 缓存行对齐；
- **局部扩展**：当主表桶满时，根据 tag 低两位选择一个扩展指针，分配新的子 bucket 并递归插入，实现局部树型扩展，有效减少全量迁移和重哈希次数；
- **全局扩容**：当某个桶的扩展桶达到 255 个（超过 8bit 限制）或主表负载因子达到设定的阈值（默认为 1.5）时发生全局扩容，容量翻倍。

其具体数据结构由表 2、表 3 及图 7 所示：

表 2 Main Table Slot 数据结构

字段	说明	字节数(Byte)
Tag	Key 的哈希摘要	2
Pointer	指向 KV 项的指针	8

表 3 Main Table Bucket 数据结构

字段	说明	字节数(Byte)
slot[]	Main Table Slot 数组，共 6 个	60
expansionPtr[]	用于解决冲突的扩展指针，共 4 个	4



图 7 Main Table 桶布局

2.3.4 Entry Store

Entry Store 负责所有状态条目的实际存储，采用如图 8 所示的键值分离设计。使用 slab 大块分配，其中值存储区采用空闲链表管理，键存储区采用写追加。既保证大吞吐分配，也降低碎片化。

- 结构设计：每个键 Entry 包含 hash、keyLen、namespaceLen、valueHandle 等元信息，支持变长 key/namespace/value；每个值 Entry 包含 valueLen 和序列化后的 value；
- 分配策略：按大块分配，写入新值时按空闲链表分配，写入新键时追加写；删除时，值空间回收，支持快速重用，而键空间不做操作；
- 对齐和效率：分配、释放均采用 8B 对齐，提升内存访问效率；
- 内存池：所有数据均在 Flink MemoryManager 分配的大块堆外内存中，无需频繁 native 调用。

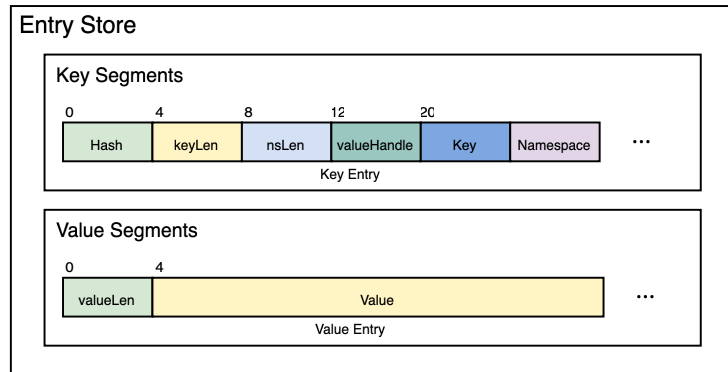


图 8 Entry Store 实现结构

其键值结构布局由表 4 和表 5 给出。

表 4 键 Entry 数据结构

字段	说明	字节数(Byte)
hash	key 与 namespace 的 hash 值	4
keyLen	key 长度	4
namespaceLen	namespace 长度	4
valueHandle	value 指针	8
key	序列化后的 key	keyLen
namespace	序列化后的 namespace	namespaceLen

表 5 值 Entry 数据结构

字段	说明	字节数(Byte)
valueLen	value 长度	2
value	序列化后的 value	8

2.3.5 主要操作流程

插入 (Put)

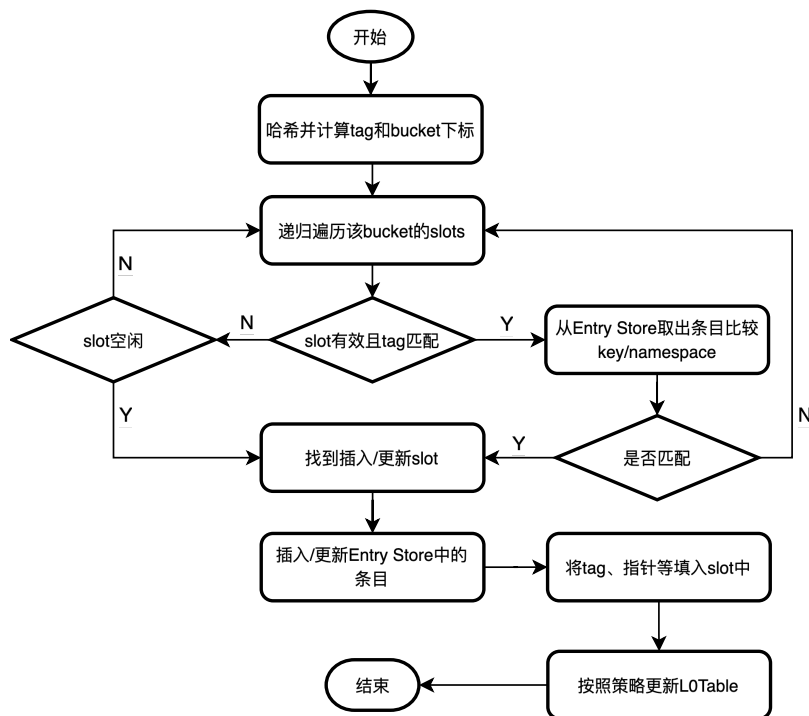


图 9 State Map 插入流程

写入流程总是先写主表，再同步到 L0 Table（由可配置替换算法决定）。其流程由图 9 所示并描述如下：

1. 计算 key 的 tag 和主表 bucket 下标。
2. 读取主表 bucket，遍历每个 slot：
 - a) 若 slot 有效且 tag 匹配，取指针指向的条目，比较 key/namespace 是否一致；
 - i. 若一致，直接更新 value 数据，提升该 slot 到 L0 Table，流程结束；
 - ii. 若不一致，继续遍历。
 - b) 若 slot 空闲，则：
 - i. 在 Entry Store 分配新的条目，写入键值；
 - ii. 将 slot 的 tag、指针等信息设置为新值；
 - iii. 更新 L0 Table，流程结束。
3. 若所有 slot 已占用，计算扩展 bucket 下标 (tag & 0x3)：
 - a) 若当前 subPtr 未分配扩展 bucket，则在扩展池中分配新 bucket，并写入 expansionPtr；
 - b) 递归步骤 2 对该扩展 bucket 进行插入操作。
4. 插入/更新后可按策略将 key 提升为 L0 热点。

查询 (Get)

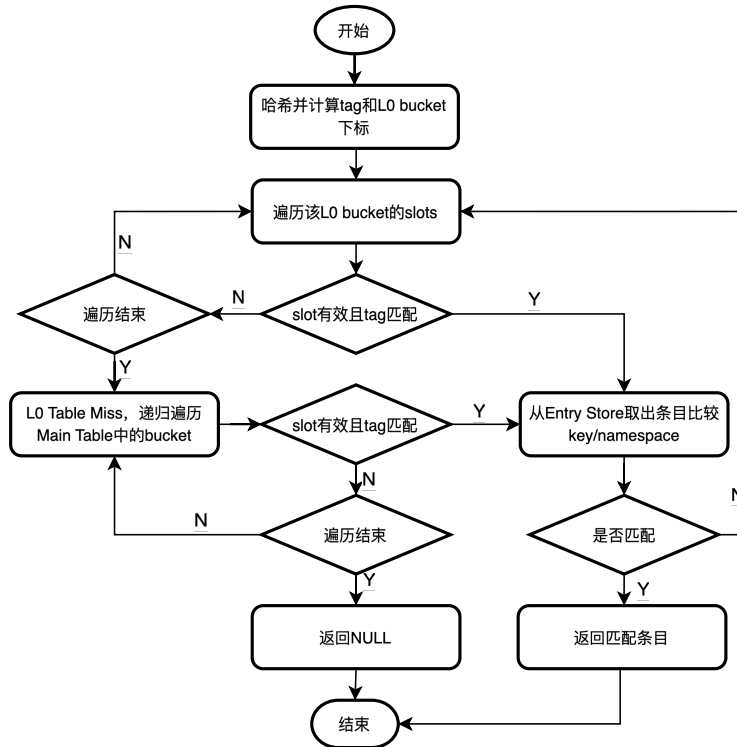


图 10 State Map 查询流程

查询流程总是优先查 L0 Table，未命中则回落主表。其流程如图 10 所示并描述如下：

1. 计算 key 的 tag (hash 高位) 和 L0 bucket 下标。
2. 读取 L0 Table 中对应 bucket，遍历其中每个 slot：
 - a) 若 slot 有效且 tag 匹配，则取出 slot 指针指向的 Entry Store 条目；
 - b) 比较的 key/namespace 数据是否与查询完全一致；
 - i. 若一致，直接返回该条目的 value 数据，流程结束；
 - ii. 若不一致，继续检查下一个 slot。
3. 若 L0 Table 未命中，则计算 Main Table 的 bucket 下标。
4. 读取主表对应 bucket，遍历 bucket 的每个 slot：
 - a) 若 slot 有效且 tag 匹配，则取出 slot 指针指向的 Entry Store 条目，比较 key/namespace 数据是否一致；
 - i. 若一致，将该项目插入或提升到 L0 Table，并返回 value 数据，流程结束；
 - ii. 若不一致，继续遍历。
5. 若主表 bucket 所有 slot 均未命中，计算扩展 bucket 下标 (tag & 0x3)，
 - a) 如果对应下标存在扩展 bucket，则递归步骤 4 对扩展 bucket 重复上述操作。
6. 若所有主表与扩展 bucket 均未命中，返回未找到。

变换 (Transform)

变换操作语义与 HeapState 保持一致，其流程描述如下：

1. 计算 key 的 tag 和主表 bucket 下标。
2. 同 Put 流程在主表中找到插入位置或更新位置。
 - a) 若为更新 (值已存在)，则直接从 Entry Store 获取值，应用变换方法后更新；
 - b) 若为插入 (值不存在)，则对空值应用变换方法后插入 Entry Store；
3. 若条目地址发生变化或为新条目，则回填主表 slot。
4. 更新 L0 Table。

删除 (Remove)

删除流程同样以主表为准，其流程描述如下：

1. 计算 key 的 tag 和主表 bucket 下标。
2. 读取主表 bucket，遍历每个 slot：
 - a) 若 slot 有效且 tag 匹配，取指针指向的条目，比较 key/namespace 数据是否一致；
 - i. 若一致，将 slot 有效位和指针清零（置为无效），并将对应条目地址加入空闲链表回收，流程继续；
 - ii. 若不一致，继续遍历。
3. 若主表 bucket 所有 slot 均未命中，计算扩展 bucket 下标 (tag & 0x3)，如有扩展 bucket，递归步骤 2。
4. 最后，找到 L0 Table 中 slot 指针指向该地址的条目，将其有效位清零。
5. 删除操作结束。

全局扩容 (Resize)

调用 Put 或 Transform 方法时会首先检查是否需要扩容，全局扩容流程描述如下：

1. 检查当前主表全局负载因子或单 bucket 扩展池数量，若超过阈值，则触发扩容。
2. 分配新主表 bucket 数组（容量为原表 2 倍），分配新的扩展 bucket 池和内存区域。
3. 递归遍历当前主表所有 bucket 及扩展 bucket：
 - a) 对于每个有效 slot，从 Entry Store 读取 hash 字段计算在新表中的下标，并插入新表。
 - b) 迁移过程中，所有条目物理地址保持不变，仅重建索引结构。
4. 完成后，用新表和新扩展池替换旧主表和旧扩展池，释放旧内存。
5. L0 Table 整体清空，待后续访问自动重建热点 key 缓存。

2.4 ForL0State 实现

ForL0StateBackend 提供与 Flink State API 一致的五类状态实现：ForL0ValueState、ForL0ListState、ForL0MapState、ForL0ReducingState 和 ForL0AggregatingState。

2.4.1 ForL0ValueState

表示每个 key/namespace 下的单一值。其主要操作与 State Map 映射关系如下：

value(): 调用底层 ForL0StateMap.get 读取当前 key/namespace 下的值；如果为 null 则返回默认值。

update(value): 当 value 不为 null 时调用 ForL0StateMap.put 写回；当 value 为 null 时底层调用 ForL0StateMap.remove。

2.4.2 ForL0ListState

表示可追加/枚举的序列集合。其主要操作与 State Map 映射关系如下：

get(): 通过底层 ForL0StateMap.get 读取当前 key/namespace 下的 List 值，若为 null 则返回空或默认值。

add(value): 先调用 ForL0StateMap.get 取出当前 List；若为 null 则新建 List，向 List 添加元素后显式调用 ForL0StateMap.put 写回，确保底层可见。

update(values): 将新的 List 完整序列化后通过 ForL0StateMap.put 覆盖写入；当 values 为空时底层调用 ForL0StateMap.remove。

addAll(values): 使用 ForL0StateMap.transform 将传入集合合并到已有状态，底层由

ForL0StateMap 的 transform 操作实现 read-modify-write 并处理可能的内存重分配。

2.4.3 ForL0MapState

表示键值映射集合。其主要操作与 State Map 映射关系如下：

get(userKey): 通过 ForL0StateMap.get 获取用户层的 Map 对象,再在该 Map 上做 userKey 的查询并返回结果。

put(userKey, userValue) / putAll(map): 先通过 ForL0StateMap.get 获取当前 Map (若 null 则新建), 在内存上修改后通过 ForL0StateMap.put 显式写回以持久化变更。

remove(userKey): 读取并修改 Map; 若修改后 Map 为空则调用 ForL0StateMap.remove 整条条目, 否则显式 put 回写以保持一致性。

contains(userKey)/entries()/keys()/values(): 均基于 ForL0StateMap.get 返回的 Map 在用户程序中进行判断或迭代, 实际的序列化/存储由 ForL0StateMap 管理。

2.4.4 ForL0ReducingState

用于增量合并的聚合状态 (reduce 聚合函数)。其主要操作与 State Map 映射关系如下：

get(): 通过 ForL0StateMap.get 读取当前累积值 (聚合结果的中间表示)。

add(value): 调用 ForL0StateMap.transform, 将新增值与已有状态通过用户提供的聚合函数合并。ForL0StateMap 的 transform 方法负责读取旧值、执行聚合转换、并将结果序列化写回 (可能为原地更新或重分配并更新主表/缓存指针)。

merge(): 合并逻辑由聚合函数提供, 最终写回仍通过 ForL0StateMap 的 put 和 transform 操作完成。

2.4.5 ForL0AggregatingState

支持更复杂的聚合语义的状态 (带 accumulator 的聚合)。其主要操作与 State Map 映射关系如下：

get(): 通过 ForL0StateMap.get 读取累加器, 若非 null 则调用 累加器的 getResult 方法得到输出值并返回。

add(value): 通过 ForL0StateMap.transform 将输入值通过累加器的 add 方法添加到累加器中, 底层通过 ForL0StateMap.transform 在 read-modify-write 一步中完成序列化与堆外存管理。

merge(): 由累加器的 merge 方法定义合并语义, 底层 ForL0StateMap 负责将合并后的累加器持久化回 Entry Store 并更新索引。

3. 容错机制设计

ForL0StateBackend 的容错设计基于 Flink 原生的快照和恢复框架, 确保与 HeapStateBackend 格式的兼容性, 并在实现上实现低干扰与高效序列化。主表 (Main Table) 用于存储全量索引, 而 L0 Table 仅作为运行时的热点索引缓存, 不参与持久化数据存储。所有实际的键值数据存储在堆外的 Entry Store 中, 主表中保存的是指向这些数据字节的指针。

在写操作 (如 put、transform、update) 中, 数据会写入 Entry Store, 并且在新增或重分配时更新主表中的指针; 如果是原地更新, 则主表中的指针保持不变, 但底层字节数据会被更新。因此, 在快照和恢复流程中, 直接以主表为基础进行序列化即可保证数据一致性。并且, 由于 ForL0StateBackend 完全使用堆外存储, 快照时能够省去大部分的序列化开销, 只需序列化一些关键的元数据 (如状态条目的相关信息等), 大大提高了快照过程的效率, 并减轻了系统负担。

3.1 快照与恢复流程

3.1.1 快照发起

在 Flink 调 `snapshot` 方法后，快照操作交由 `SnapshotStrategyRunner` 执行，具体同步或异步执行由配置的 `SnapshotExecutionType` 控制。已注册的快照策略会处理整个快照流程，默认采用与 `HeapStateBackend` 兼容的实现。

之后，`stateSnapshot` 方法为每个 `key-group` 生成状态快照，并收集对应 `key-group` 下的 `StateMap` 快照列表。每个 `ForL0StateMap` 快照会以与 `HeapStateBackend` 相同的格式提供序列化接口，确保与 Flink 的 `Checkpoint` 和 `Savepoint` 格式兼容。

3.1.2 快照采集

`ForL0StateMapSnapshot` 通过遍历 `Main Table` 的条目，从 `Entry Store` 读取 `key`、`namespace` 和对应的 `value` 字节，构建一个有效的条目列表。随后，快照数据会按 Flink 要求的顺序被写入到 `Checkpoint` 输出流中，包括条目数及每条记录的格式（`namespace`、`key`、`state`）。

这个实现将遍历与实际序列化写入分为两步：第一步快速收集有效条目，第二步再进行序列化写入，从而减少对正常读写的影响。在此过程中，可选的 `StateSnapshotTransformer` 可以应用于写入前，以支持状态转换和裁剪。

3.2 Savepoint 与恢复（Restore）

`ForL0KeyedStateBackend` 中的 `savepoint` 方法通过与 `HeapKeyedStateBackend` 后端相同的 `HeapSnapshotResources` 生成保存点资源，并按照标准 Flink `Savepoint` 格式输出，便于手动触发的状态迁移或升级操作。

而 `ForL0RestoreOperation` 继承并复用了 `HeapRestoreOperation` 的恢复逻辑，它们使用相同的数据格式。在恢复过程中，系统会按 `key-group` 重建相关堆外数据结构。恢复后，`L0 Cache` 为空，运行时会根据访问情况自动回填热点索引。

恢复流程遵循 Flink 的一致性语义，确保从持久化的 `KeyedStateHandle` 中逐条重建 `Main Table` 与 `Entry Store` 数据结构。

3.3 资源回收与异常处理

`ForL0StateMapSnapshot` 和 `ForL0StateTableSnapshot` 提供 `release` 方法，在快照完成或取消后清理临时引用，帮助垃圾回收释放堆外和堆内资源。

对于快照写入过程中发生的序列化或 IO 异常，快照逻辑会根据 Flink 约定向上抛出异常，上层调用者会负责重试或失败处理。在写入前，系统会做必要的序列化器复制和输入校验，以尽早发现不兼容问题。

4. 插件式集成设计

为确保易用性与兼容性，`ForL0StateBackend` 采用插件化实现方式，通过 Flink 的 `StateBackendFactory` 接口进行适配。该设计满足以下目标：

- **无侵入接入**：无需修改 Flink 内核源码与用户算子代码；
- **配置可控**：通过集群配置（`config.yaml` / `flink-conf.yaml`）或用户代码显式设置启用；
- **与现有作业兼容**：对 Flink 状态 API 保持透明。

4.1 加载机制

ForL0StateBackend 采用与 Flink 官方 RocksDBStateBackend 相同的加载机制，通过工厂接口解析配置并创建 ForL0StateBackend 实例，其运行时的动态流程如图 11 所示。

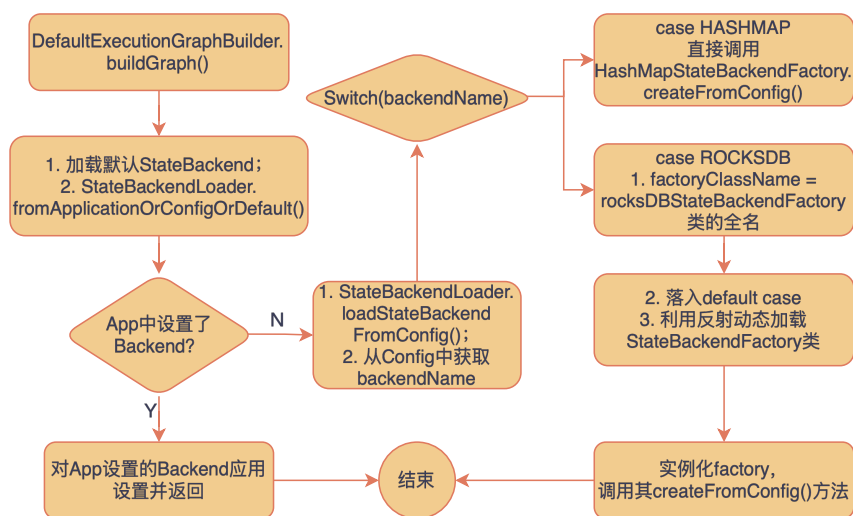


图 11 Flink 运行时 StateBackend 动态加载流程

随后，Flink 会基于该后端为各算子构建 `KeyedStateBackend` 与 `OperatorStateBackend` 并创建和初始化相应状态。

4.2 打包与部署

编译 ForL0StateBackend 项目后获得产物 `forl0-statebackend-<version>.jar`，其包含上述设计中所有组件的实现，将生成 Jar 包放入 Flink 指定库目录下后即可配置使用。

通过配置文件：由于本后端基于 `HeapKeyedStateBackend` 进行改造，需要保持与之相同的路径以获得相同的访问权限。因此，需在集群或作业的配置文件中指定 “`state.backend: org.apache.runtime.state.heap.ForL0StateBackendFactory`” 进行使用。这是推荐的使用方式，因为放置 JAR 后仅通过配置即可启用，无需变更用户代码。

通过用户代码：当以依赖方式引入本状态后端时，可以在作业中设置 `ForL0StateBackend` 为状态后端，使用方法与其它后端相同，具体设置方式请参阅 Flink 官方文档。

5. 总结

ForL0StateBackend 以两层索引结构最大化利用鲲鹏 CPU L0 缓存特性，显著降低状态访问延迟并提升吞吐为设计目标。系统由 L0 Table（热点索引缓存层）与 Main Table（全量索引层）及 Entry Store（堆外 KV 存储）构成：L0 Table 常驻 L0 缓存区域、用于加速高频访问；Main Table 保存全部键的指针并采用缓存友好的哈希与局部扩展策略；实际键值字节保存在 Entry Store，堆外内存通过利用 Flink 的 `MemoryManager` 实现的自定义分配器统一分配与管理。容错方面与 Flink 原生快照机制兼容：快照基于 Main Table 与 Entry Store 的条目遍历与标准序列化格式生成状态快照，支持 `Savepoint/Restore` 功能，并通过快速收集条目与延后序列化的策略将对在线读写的影响降到最低。总体上，设计在保持与 Flink 快照格式和运行时集成兼容性的同时，实现了充分利用 L0 缓存的、硬件亲和的低延迟、高吞吐的堆外状态后端。